

Gemini 3.8 Flash is now available. [Try it out](#)

([https://aistudio.google.com/prompts/new\\_chat?model=gemini-3.8-flash](https://aistudio.google.com/prompts/new_chat?model=gemini-3.8-flash)).

# Gemini 3 developer guide

Gemini 3 is our most intelligent model family to date, built on a foundation of state-of-the-art reasoning. It is designed to bring any idea to life by mastering agentic workflows, autonomous coding, and complex multimodal tasks. This guide covers key features of the Gemini 3 model family and how to get the most out of it.

Explore our [collection of Gemini 3 apps](#)

(<https://aistudio.google.com/app/apps?source=showcase&showcaseTag=gemini-3>) to see how the model handles advanced reasoning, autonomous coding, and complex multimodal tasks.

Get started with a few lines of code:

**Python** **JavaScript** (#javascript) **Java** (#java) **Go** (#go) **REST** (#rest)  
(#python)

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.1-pro-preview",
    input="Find the race condition in this multi-threaded C++ snippet:
[code here]",
)

print(interaction.output_text)
```

## Meet the Gemini 3 series

Gemini 3.1 Pro is best for complex tasks that require broad world knowledge and advanced reasoning across modalities.

Gemini 3 Flash is our latest 3-series model, with Pro-level intelligence at the speed and pricing of Flash.

Nano Banana Pro (also known as Gemini 3 Pro Image) is our highest quality image generation model, and Nano Banana 2 (also known as Gemini 3.1 Flash Image) is the high-volume, high-efficiency, lower price-point equivalent.

Gemini 3.1 Flash-Lite is our workhorse model built for cost-efficiency model and high-volume tasks.

All Gemini 3 models are currently in preview.

| Model ID                       | Context Window (In / Out) | Knowledge Cutoff | Pricing (Input / Output)*                              |
|--------------------------------|---------------------------|------------------|--------------------------------------------------------|
| gemini-3.1-flash-lite          | 1M / 64k                  | Jan 2025         | \$0.25 (text, image, video), \$0.50 (audio) / \$1.50   |
| gemini-3.1-flash-image-preview | 128k / 32k                | Jan 2025         | \$0.25 (Text Input) / \$0.067 (Image Output)**         |
| gemini-3.1-pro-preview         | 1M / 64k                  | Jan 2025         | \$2 / \$12 (<200k tokens)<br>\$4 / \$18 (>200k tokens) |
| gemini-3-flash-preview         | 1M / 64k                  | Jan 2025         | \$0.50 / \$3                                           |
| gemini-3-pro-image-preview     | 65k / 32k                 | Jan 2025         | \$2 (Text Input) / \$0.134 (Image Output)**            |

*\* Pricing is per 1 million tokens unless otherwise noted. \*\* Image pricing varies by resolution. See the [pricing page](/gemini-api/docs/pricing) (/gemini-api/docs/pricing) for details.*

For detailed limits, pricing, and additional information, see the [models page](/gemini-api/docs/models/gemini) (/gemini-api/docs/models/gemini).

## New API features in Gemini 3

Gemini 3 introduces new parameters designed to give developers more control over latency, cost, and multimodal fidelity.

### Level of thinking

Gemini 3 series models use dynamic thinking by default to reason through prompts. You can use the `thinking_level` parameter, which controls the **maximum** depth of the model's internal reasoning process before it produces a response. Gemini 3 treats these levels as relative allowances for thinking rather than strict token guarantees.

If `thinking_level` is not specified, Gemini 3 will default to `high`. For faster, lower-latency responses when complex reasoning isn't required, you can constrain the model's thinking level to `low`.

| Thinking Level | Gemini 3.1 Pro               | Gemini 3.1 Flash-Lite | Gemini 3 Flash               | Description                                                                                                                                                                                                                                |
|----------------|------------------------------|-----------------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| minimal        | Not supported                | Supported (Default)   | Supported                    | Matches the "no thinking" setting for most queries. The model may think very minimally for complex coding tasks. Minimizes latency for chat or high throughput applications. Note, <b>minimal</b> does not guarantee that thinking is off. |
| low            | Supported                    | Supported             | Supported                    | Minimizes latency and cost. Best for simple instruction following, chat, or high-throughput applications.                                                                                                                                  |
| medium         | Supported                    | Supported             | Supported                    | Balanced thinking for most tasks.                                                                                                                                                                                                          |
| high           | Supported (Default, Dynamic) | Supported (Dynamic)   | Supported (Default, Dynamic) | Maximizes reasoning depth. The model may take significantly longer to reach a first (non thinking) output token, but the output will be more carefully reasoned.                                                                           |

```
from google import genai

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.1-pro-preview",
    input="How does AI work?",
    generation_config={"thinking_level": "low"},
)

print(interaction.output_text)
```

## Temperature

For all Gemini 3 models, we strongly recommend keeping the temperature parameter at its default value of **1.0**.

While previous models often benefited from tuning temperature to control creativity versus determinism, Gemini 3's reasoning capabilities are optimized for the default setting. Changing the temperature (setting it below 1.0) may lead to unexpected behavior, such as looping or degraded performance, particularly in complex mathematical or reasoning tasks.

## Thought signatures

Gemini 3 models use thought signatures to maintain reasoning context across API calls. These signatures are encrypted representations of the model's internal thought process.

- **Stateful Mode (Recommended):** When using the Interactions API in stateful mode (providing `previous_interaction_id`), the server automatically manages conversation history and thought signatures.
- **Stateless Mode:** If you are managing conversation history manually, you must include thought blocks with their signatures in subsequent requests to validate authenticity.

For detailed information, see the [Thought Signatures \(/gemini-api/docs/thinking\)](/gemini-api/docs/thinking) page.

## Structured Outputs with tools

Gemini 3 models allow you to combine [Structured Outputs \(/gemini-api/docs/structured-output\)](/gemini-api/docs/structured-output) with built-in tools, including [Grounding with Google Search \(/gemini-api/docs/google-search\)](/gemini-api/docs/google-search),

[URL Context](/gemini-api/docs/url-context) (/gemini-api/docs/url-context), [Code Execution](/gemini-api/docs/code-execution) (/gemini-api/docs/code-execution), and [Function Calling](/gemini-api/docs/function-calling) (/gemini-api/docs/function-calling).

[Python](#)JavaScript (#javascript)[Java](#) (#java)[Go](#) (#go)[REST](#) (#rest)  
(#python)

```
from google import genai
from pydantic import BaseModel, Field
from typing import List

class MatchResult(BaseModel):
    winner: str = Field(description="The name of the winner.")
    final_match_score: str = Field(description="The final match score.")
    scorers: List[str] = Field(description="The name of the scorer.")

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3.1-pro-preview",
    input="Search for all details for the latest Euro.",
    tools=[
        {"type": "google_search"},
        {"type": "url_context"}
    ],
    response_format={
        "type": "text",
        "mime_type": "application/json",
        "schema": MatchResult.model_json_schema()
    },
)

result = MatchResult.model_validate_json(interaction.output_text)
print(result)
```

## Image generation

Gemini 3.1 Flash Image and Gemini 3 Pro Image let you generate and edit images from text prompts. It uses reasoning to "think" through a prompt and can retrieve real-time data—such as weather forecasts or stock charts—before using [Google Search](/gemini-api/docs/google-search) (/gemini-api/docs/google-search) grounding before generating high-fidelity images.

### New & improved capabilities:

- **4K & text rendering:** Generate sharp, legible text and diagrams with up to 2K and 4K resolutions.

- **Grounded generation:** Use the `google_search` tool to verify facts and generate imagery based on real-world information. Grounding with Google *Image* Search available for Gemini 3.1 Flash Image.
- **Conversational editing:** Multi-turn image editing by simply asking for changes (e.g., "Make the background a sunset"). This workflow relies on **Thought Signatures** to preserve visual context between turns.

For complete details on aspect ratios, editing workflows, and configuration options, see the [Image Generation guide](/gemini-api/docs/image-generation) (/gemini-api/docs/image-generation).

`PythonJavaScript` (#javascript)`Java` (#java)`Go` (#go)`REST` (#rest)  
(#python)

```
from google import genai
import base64

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-pro-image-preview",
    input="Generate an infographic of the current weather in Tokyo.",
    tools=[{"type": "google_search"}],
    response_format={
        "type": "image",
        "aspect_ratio": "16:9",
        "image_size": "4K"
    }
)

from PIL import Image
import io

generated_image = interaction.output_image
if generated_image:
    image_data = base64.b64decode(generated_image.data)
    image = Image.open(io.BytesIO(image_data))
    image.save('weather_tokyo.png')
    image.show()
```

## Example Response



## Code Execution with images

Gemini 3 Flash can treat vision as an active investigation, not just a static glance. By combining reasoning with [code execution](#) (/gemini-api/docs/code-execution), the model formulates a plan, then writes and executes Python code to zoom in, crop, annotate, or otherwise manipulate images step-by-step to visually ground its answers.

### Use cases:

- **Zoom and inspect:** The model implicitly detects when details are too small (e.g., reading a distant gauge or serial number) and writes code to crop and re-examine the area at higher resolution.
- **Visual math and plotting:** The model can run multi-step calculations using code (e.g., summing line items on a receipt, or generating a Matplotlib chart from extracted data).
- **Image annotation:** The model can draw arrows, bounding boxes, or other annotations directly onto images to answer spatial questions like "Where should this item go?".

To enable visual thinking, configure [Code Execution](#) (/gemini-api/docs/code-execution) as a tool. The model will automatically use code to manipulate images when needed.

```
PythonJavaScript (#javascript)Java (#java)Go (#go)REST (#rest)
(#python)
```

```
from google import genai
from google.genai import types
```

```

import requests
from PIL import Image
import io
import base64

image_path = "https://goo.gle/instrument-img"
image_bytes = requests.get(image_path).content
image = types.Part.from_bytes(data=image_bytes, mime_type="image/jpeg")

client = genai.Client()

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input=[
        image,
        "Zoom into the expression pedals and tell me how many pedals are there?"
    ],
    tools=[{"type": "code_execution"}],
)

from IPython.display import display
from PIL import Image
import io

for step in interaction.steps:
    if step.type == "model_output":
        for content_block in step.content:
            if content_block.type == "text":
                print(content_block.text)
            elif content_block.type == "image":

display(Image.open(io.BytesIO(base64.b64decode(content_block.data))))
    elif step.type == "code_execution_call":
        print(step.code)
    elif step.type == "code_execution_result":
        print(step.output)

```

For more details on code execution with images, see [Code Execution \(/gemini-api/docs/code-execution#images\)](/gemini-api/docs/code-execution#images).

## Multimodal function responses

[Multimodal function calling \(/gemini-api/docs/function-calling#multimodal\)](/gemini-api/docs/function-calling#multimodal) allows users to have function responses containing multimodal objects allowing for improved utilization of

function calling capabilities of the model. Standard function calling only supports text-based function responses:

PythonJavaScript (#javascript)Java (#java)Go (#go)REST (#rest)  
(#python)

```
# This will only work for SDK newer than 2.0.0
from google import genai
import requests
import base64

client = genai.Client()

# 1. Define the tool
get_image_tool = {
    "type": "function",
    "name": "get_image",
    "description": "Retrieves the image file reference for a specific
order item.",
    "parameters": {
        "type": "object",
        "properties": {
            "item_name": {
                "type": "string",
                "description": "The name or description of the item
ordered (e.g., 'instrument')."
            }
        },
        "required": ["item_name"],
    },
}

# 2. Send the request with tools
interaction_1 = client.interactions.create(
    model="gemini-3-flash-preview",
    input="Show me the instrument I ordered last month.",
    tools=[get_image_tool],
)

# 3. Find the function call step
fc_step = next(s for s in interaction_1.steps if s.type ==
"function_call")
print(f"Tool Call: {fc_step.name}({fc_step.arguments})")

# Execute tool (fetch image)
image_path = "https://goo.gle/instrument-img"
image_bytes = requests.get(image_path).content
image_b64 = base64.b64encode(image_bytes).decode("utf-8")
```

```
# 4. Send multimodal function result back
interaction_2 = client.interactions.create(
    model="gemini-3-flash-preview",
    previous_interaction_id=interaction_1.id,
    input=[{
        "type": "function_result",
        "name": fc_step.name,
        "call_id": fc_step.id,
        "result": [
            {"type": "text", "text": "instrument.jpg"},
            {
                "type": "image",
                "mime_type": "image/jpeg",
                "data": image_b64,
            }
        ]
    }],
    tools=[get_image_tool]
)

print(f"\nFinal model response: {interaction_2.output_text}")
```

## Combine built-in tools and function calling

Gemini 3 allows the use of built-in tools (like Google Search, URL context, and [more \(/gemini-api/docs/tools\)](/gemini-api/docs/tools)) and custom [function calling \(/gemini-api/docs/function-calling\)](/gemini-api/docs/function-calling) tools in the same API call, allowing for more complex workflows.

**Python**JavaScript (#javascript)**Java** (#java)**Go** (#go)  
(#python)

```
from google import genai
from google.genai import types

client = genai.Client()

getWeather = {
    "type": "function",
    "name": "getWeather",
    "description": "Gets the weather for a requested city.",
    "parameters": {
        "type": "object",
        "properties": {
            "city": {
```

```

        "type": "string",
        "description": "The city and state, e.g. Utqiaġvik,
Alaska",
    },
    },
    "required": ["city"],
},
}

interaction = client.interactions.create(
    model="gemini-3-flash-preview",
    input="What is the northernmost city in the United States? What's
the weather like there today?",
    tools=[
        {"type": "google_search"},
        getWeather
    ],
)

fc_step = next((s for s in interaction.steps if s.type ==
"function_call"), None)

if fc_step:
    result = {"response": "Very cold. 22 degrees Fahrenheit."}

    final_interaction = client.interactions.create(
        model="gemini-3-flash-preview",
        input=[
            {"type": "function_result", "name": fc_step.name, "call_id":
fc_step.id, "result": result}
        ],
        tools=[
            {"type": "google_search"},
            getWeather
        ],
        previous_interaction_id=interaction.id,
    )

    print(final_interaction.output_text)

```

## Migration from Gemini 2.5

Gemini 3 is our most capable model family to date and offers a stepwise improvement over Gemini 2.5. When migrating, consider the following:

- **Thinking:** If you were previously using complex prompt engineering (like chain of thought) to force Gemini 2.5 to reason, try Gemini 3 with `thinking_level: "high"` and simplified prompts.
- **Temperature settings:** If your existing code explicitly sets temperature (especially to low values for deterministic outputs), we recommend removing this parameter and using the Gemini 3 default of 1.0 to avoid potential looping issues or performance degradation on complex tasks.
- **PDF & document understanding:** If you relied on specific behavior for dense document parsing, test the new `media_resolution_high` setting to ensure continued accuracy.
- **Token consumption:** Migrating to Gemini 3 defaults may **increase** token usage for PDFs but **decrease** token usage for video. If requests now exceed the context window due to higher default resolutions, we recommend explicitly reducing the media resolution.
- **Image segmentation:** Image segmentation capabilities (returning pixel-level masks for objects) are not supported in Gemini 3 Pro or Gemini 3 Flash. For workloads requiring built-in image segmentation, we recommend continuing to use Gemini 2.5 Flash with thinking turned off.
- **Computer Use:** Gemini 3 Pro and Gemini 3 Flash support Computer Use (</gemini-api/docs/computer-use>). Unlike the 2.5 series, you don't need to use a separate model to access the Computer Use tool.
- **Tool support:** Combining built-in tools with function calling (</gemini-api/docs/tool-combination>) is now supported for Gemini 3 models. Maps grounding (</gemini-api/docs/maps-grounding>) is also now supported for Gemini 3 models.

## OpenAI compatibility

For users utilizing the OpenAI compatibility layer (</gemini-api/docs/openai>), standard parameters (OpenAI's `reasoning_effort`) are automatically mapped to Gemini (`thinking_level`) equivalents.

## Prompting best practices

Gemini 3 is a reasoning model, which changes how you should prompt.

- **Precise instructions:** Be concise in your input prompts. Gemini 3 responds best to direct, clear instructions. It may over-analyze verbose or overly complex prompt engineering techniques used for older models.
- **Output verbosity:** By default, Gemini 3 is less verbose and prefers providing direct, efficient answers. If your use case requires a more conversational or "chatty" persona, you must explicitly steer the model in the prompt (e.g., "Explain this as a friendly, talkative assistant").
- **Context management:** When working with large datasets (e.g., entire books, codebases, or long videos), place your specific instructions or questions at the end of the prompt, after the data context. Anchor the model's reasoning to the provided data by starting your question with a phrase like, "Based on the preceding information..."

Learn more about prompt design strategies in the [prompt engineering guide](/gemini-api/docs/prompting-strategies) (/gemini-api/docs/prompting-strategies).

## FAQ

1. **What is the knowledge cutoff for Gemini 3?** Gemini 3 models have a knowledge cutoff of January 2025. For more recent information, use the [Search Grounding](/gemini-api/docs/google-search) (/gemini-api/docs/google-search) tool.
2. **What are the context window limits?** Gemini 3 models support a 1 million token input context window and up to 64k tokens of output.
3. **Is there a free tier for Gemini 3?** Gemini 3 Flash `gemini-3-flash-preview` has a free tier in the Gemini API. You can try Gemini 3.1 Pro and 3 Flash at no cost in Google AI Studio, but there is no free tier available for `gemini-3.1-pro-preview` in the Gemini API.
4. **Will my old `thinking_budget` code still work?** Yes, `thinking_budget` is still supported for backward compatibility, but we recommend migrating to `thinking_level` for more predictable performance. Don't use both in the same request.
5. **Does Gemini 3 support the Batch API?** Yes, Gemini 3 supports the [Batch API](/gemini-api/docs/batch-api) (/gemini-api/docs/batch-api).
6. **Is Context Caching supported?** Yes, [Context Caching](/gemini-api/docs/caching) (/gemini-api/docs/caching) is supported for Gemini 3.
7. **Which tools are supported in Gemini 3?** Gemini 3 supports [Google Search](/gemini-api/docs/google-search) (/gemini-api/docs/google-search), [Grounding with Google Maps](#)

(</gemini-api/docs/maps-grounding>), [File Search](/gemini-api/docs/file-search) (</gemini-api/docs/file-search>), [Code Execution](/gemini-api/docs/code-execution) (</gemini-api/docs/code-execution>), and [URL Context](/gemini-api/docs/url-context) (</gemini-api/docs/url-context>). It also supports standard [Function Calling](/gemini-api/docs/function-calling) (</gemini-api/docs/function-calling>) for your own custom tools, and in [combination with built-in tools](/gemini-api/docs/tool-combination) (</gemini-api/docs/tool-combination>).

8. **What is `gemini-3.1-pro-preview-customtools`?** If you are using `gemini-3.1-pro-preview` and the model ignores your custom tools in favor of bash commands, try the `gemini-3.1-pro-preview-customtools` model instead. More info [\[here\]](#) [\[customtools-model\]](#).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-09-23 UTC.